PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Учебная ситуация

Содержание

- Команды pandas, которые понадобятся
- Практические задания
- Аккуратная очистка
- Итог урока

Команда готовит таблицу о школьниках для анализа учебной нагрузки и самочувствия. В таблице есть возраст, сон, экранное время, применение ИИ для учёбы, уровень стресса, подготовка к экзаменам и итоговая группа наблюдения. Перед выводами таблицу нужно проверить.

Такая проверка необходима до вычисления средних значений, сравнения групп и обучения модели ИИ. Даже одна ошибочная строка может заметно повлиять на результат, особенно если таблица небольшая. Например, возраст 150 лет увеличит средний возраст, а повтор одной строки придаст данным об одном школьнике двойной вес.

Таблицу с пропусками, дублями, опечатками и недопустимыми значениями часто называют **грязными данными**. Очистка данных состоит не в произвольном исправлении ячеек, а в последовательном поиске проблем и применении заранее определённых правил.

Тип проблемы	Пример из таблицы	Почему мешает
Пропуск	пустое значение sleep_hours, age или screen_time_hours	часть информации об объекте неизвестна
Дубль	повтор строки с student_id = 3	один объект начинает влиять на анализ дважды
Странное число	age = 150, screen_time_hours = -2, sleep_hours = 24	статистика и выводы искажаются
Опечатка категории	medum вместо medium, hight вместо high	одна категория превращается в несколько разных

У разных проблем разные способы обнаружения. Пропуск определяется как отсутствие значения, полный дубль — как повтор всей строки, а странное число выявляется проверкой допустимого диапазона. Опечатки в категориях удобно искать по списку уникальных значений. Важно различать пропуск и ошибочное значение. Например, пустая ячейка age означает, что возраст неизвестен, а age = 150 содержит число, но оно не соответствует правилам рассматриваемого набора данных. Поэтому обычный подсчёт пропусков не обнаружит возраст 150.

Команды pandas, которые понадобятся

Для проверки используется библиотека pandas. После чтения CSV-файла данные сохраняются в объекте DataFrame — таблице со строками и именованными столбцами.

```
import pandas as pd
```

```
df = pd.read_csv('student_wellness_dirty_1_13.csv')
```

```
df.isna().sum()
```

```
df.duplicated().sum()
```

```
df['stress_level'].unique()
```

```
df.drop_duplicates()
```

```
df['stress_level'].replace({'medum':'medium'})
```

Команды решают разные задачи:

- **df.isna()** отмечает пропущенные ячейки значениями True, а последовательные вызовы **sum()** позволяют подсчитать их;
- **df.duplicated()** отмечает строки, которые полностью повторяют уже встречавшиеся строки;
- **unique()** возвращает набор различных значений выбранного столбца и помогает заметить лишние варианты написания;
- **drop_duplicates()** создаёт таблицу без повторяющихся строк;
- **replace()** заменяет значения по указанному словарю соответствий.

Большинство операций pandas в этих примерах возвращают новый результат, но сами по себе не изменяют исходный DataFrame. Поэтому результат очистки нужно присвоить переменной или столбцу. Например, вызов df.drop_duplicates() позволяет увидеть таблицу без дублей, а запись clean = df.drop_duplicates().copy() сохраняет очищенную таблицу в переменной clean. При подсчёте важно следить за количеством вызовов sum(). Выражение df.isna().sum() показывает число пропусков отдельно в каждом столбце, а df.isna().sum().sum() даёт одно число — общее количество пропущенных ячеек во всей таблице.

Практические задания

В этом уроке задания разделены на уровень 1 и уровень 2. Уровень 1 выполняется по образцу. Уровень 2 выполняется в изменённых условиях: нужно применить похожую команду к другому столбцу или объединить несколько действий.
Перед выполнением заданий полезно сначала загрузить исходный файл в переменную df и не перезаписывать её. Тогда результаты до очистки можно будет сравнить с таблицей clean после очистки. Такое сравнение помогает убедиться, что нужные строки или значения действительно изменились, а остальные данные сохранились.

№	Уровень	Название	Что должна вывести программа
1	1	Размер грязной таблицы	15 8
2	1	Общее число пропусков	3
3	2	Количество полных дублей	1
4	2	Максимальный возраст	150
5	2	Отрицательное экранное время	1

№	Уровень	Название	Что должна вывести программа
6	2	Категории уровня стресса до очистки	high,hight,low,medium,medum
7	2	Исправление категорий stress_level	high,low,medium
8	2	Удаление дублей	14 8
9	2	Полная базовая очистка по правилам	6

Размер таблицы задаётся двумя числами: количеством строк и количеством столбцов. Поэтому результат 15 8 означает, что в исходной таблице 15 строк и 8 столбцов. После удаления одного полного дубля ожидаемый размер становится 14 8: число строк уменьшается, а набор столбцов остаётся прежним.
Задания на максимальный возраст и отрицательное экранное время проверяют не пропуски, а логические ограничения. Максимум помогает быстро заметить слишком большое значение, а условие screen_time_hours < 0 выделяет значения, которые не могут обозначать продолжительность.
При проверке категорий порядок значений может зависеть от способа их получения и сортировки. Существенно, чтобы до очистки были обнаружены варианты high, hight, low, medium и medum, а после замен остались только high, low и medium.

Пример: проверяем пропуски

```
import pandas as pd
df = pd.read_csv('student_wellness_dirty_1_13.csv')
print(df.isna().sum().sum())
```

Если программа выводит 3, значит в таблице найдено три пропущенных значения. Важно не только получить число, но и понять смысл: три ячейки не содержат информации, поэтому дальнейший анализ без проверки может быть неточным.
Общее число не показывает, в каких именно столбцах находятся пропуски. Для уточнения можно отдельно рассмотреть результат df.isna().sum(): он содержит количество пустых ячеек для каждого столбца. Это помогает выбрать правило обработки с учётом смысла конкретного признака.
Частая ошибка — считать, что число 3 означает три неполные строки. На самом деле подсчитываются ячейки. Несколько пропусков могут находиться как в трёх разных строках, так

и в одной строке.
Мини-проверка: какой результат нужен для поиска проблемного столбца — общее число df.isna().sum().sum() или подсчёт по столбцам df.isna().sum()? Для поиска столбца нужен второй вариант.

Аккуратная очистка

Очистка не означает, что нужно угадывать правильные значения. Если возраст указан как 150, нельзя просто заменить его на 15 без источника. Надёжнее пометить такое значение как неизвестное. Очевидные опечатки категорий можно исправить по правилу: medum → medium, hight → high.
Перед заменой необходимо сформулировать правило. Для возраста в примере допустимым считается диапазон от 10 до 25 лет включительно. Значения меньше 10 и больше 25 помечаются как неизвестные, а граничные значения 10 и 25 сохраняются.

```
clean = df.drop_duplicates().copy()
clean['stress_level'] = clean['stress_level'].replace({'medum':'medium', 'hight':'high'})
clean.loc[(clean['age'] < 10) | (clean['age'] > 25), 'age'] = pd.NA
```

Очистка выполняется последовательно. Сначала удаляются полные дубли и создаётся независимая копия таблицы clean. Затем в столбце stress_level исправляются два известных варианта опечаток. После этого loc выбирает строки с возрастом вне допустимого диапазона и записывает pd.NA только в столбец age этих строк.
Знак | в условии означает «или»: значение считается проблемным, если возраст меньше 10 или больше 25. Каждая часть условия заключена в скобки, потому что pandas сравнивает значения столбца поэлементно.
Замена на pd.NA не восстанавливает настоящий возраст школьника. Она честно фиксирует, что после проверки корректное значение неизвестно. В дальнейшем такие ячейки можно отдельно подсчитать и учесть при анализе.
Не следует удалять строку только потому, что в одной ячейке обнаружена проблема: вместе с ней будет потеряна корректная информация из остальных столбцов. Удаление строки допустимо лишь тогда, когда это предусмотрено правилом анализа. В данном примере сомнительный возраст заменяется пропуском, а сама строка сохраняется.
После очистки нужна повторная проверка: следует снова посмотреть размер таблицы, число дублей, уникальные категории stress_level и значения, выходящие за допустимые диапазоны. Очистка считается завершённой не после выполнения команд, а после подтверждения, что правила сработали ожидаемым образом.

Итог урока

- грязная таблица может дать неверный анализ;
- пропуски, дубли, опечатки и странные числа ищутся разными командами;
- пропуск и недопустимое значение — разные виды проблем и требуют разных проверок;
- очистка должна идти по понятным правилам;
- исходную таблицу полезно сохранять отдельно, чтобы сравнивать данные до и после обработки;
- после исправлений необходимо повторно проверить размер, пропуски, дубли, категории и числовые диапазоны;
- модель ИИ учится на тех данных, которые ей дали; если данные плохие, результат тоже может быть ненадёжным.

Базовый порядок работы с данными выглядит так: загрузить таблицу, изучить её размер и содержимое, найти проблемы, применить обоснованные правила очистки и проверить результат. Такой порядок делает изменения воспроизводимыми: другой участник команды сможет понять, какие значения были изменены и почему.

Материалы к занятию

- lesson_1_13_data_quality_reworked.pptx (45.2 КБ)

Зачем нужен этот практикум

Содержание

- Цели практикума
- Материалы к практике

В предыдущем уроке таблица рассматривалась как датасет: строки описывали объекты, столбцы хранили свойства и возможный результат. Теперь важен следующий шаг: перед анализом таблицу нужно проверить. Данные могут выглядеть аккуратно, но внутри часто есть пропуски, повторы, опечатки и неправдоподобные числа.

Такая проверка похожа на просмотр заполненных анкет перед подсчётом ответов. Если один человек случайно записан дважды, часть полей не заполнена, а одна и та же категория указана в разных вариантах, итоговые расчёты могут оказаться неточными. Поэтому сначала оценивается качество исходных данных и только затем выполняется их анализ.

Разные проблемы требуют разных проверок. Пропуск означает, что значение отсутствует; дубль — что одна и та же запись повторяется; ошибка категории возникает, когда равные по смыслу значения записаны неодинаково. Странным числом может быть значение, которое не соответствует допустимому диапазону или заметно отличается от остальных наблюдений. При этом необычное значение нельзя автоматически считать ошибкой: сначала необходимо проверить правило, по которому оно оценивается.

На этом занятии работа строится через короткие программы. Каждое практическое действие должно дать проверяемый результат: число, строку категорий или новый размер таблицы. Так код можно проверить автоматически на платформе.

Например, поиск пропусков может завершаться числом найденных пустых значений, проверка категорий — списком или строкой уникальных вариантов, а удаление повторов — изменением количества строк. Такой результат позволяет убедиться, что программа не только выполнилась без ошибки, но и решила требуемую задачу.

Важно: очистка данных должна выполняться по понятным и воспроизводимым правилам. Нельзя удалять или заменять значение только потому, что оно кажется необычным: решение должно опираться на условие задачи, допустимый диапазон или единый формат записи.

Цели практикума

- открыть CSV-файл с учебной грязной таблицей и убедиться, что данные прочитаны в ожидаемом виде;
- проверить размер таблицы, названия столбцов и типы представленных в них значений;
- найти пропуски, дубли, ошибки категорий и странные числа;
- различать отсутствие значения, повтор записи и несогласованный вариант написания категории;
- исправить простые проблемы по понятным правилам и проверить результат после каждого изменения;
- сравнить количество строк или набор значений до и после очистки;
- получить таблицу, более пригодную для дальнейшего анализа;
- понять, почему качество данных влияет на будущую работу ИИ: модель обучается на переданных ей примерах, поэтому ошибки и несогласованности в них могут повлиять на найденные закономерности.

Во время практикума полезно разделять обнаружение и исправление проблемы. Сначала программа должна показать, что именно найдено, а затем отдельное действие меняет данные. После исправления проверка запускается повторно: это помогает убедиться, что проблема устранена и другие записи не были изменены случайно.

Материалы к практике

Для работы понадобятся ноутбук с заданиями и два CSV-файла. Исходная грязная таблица используется для поиска проблем, а частично очищенная версия — для следующих этапов проверки и обработки. Файлы следует сохранить с исходными именами, чтобы пути к ним в учебном коде оставались корректными.

- lesson_1_13_data_quality_start.ipynb (7.7 КБ)
- student_wellness_dirty_1_13.csv (708 Б)
- student_wellness_partly_clean_1_13.csv (660 Б)

Перед началом работы необходимо проверить, что оба CSV-файла доступны из ноутбука. Если файл не найден, программа не сможет открыть таблицу, а проверка качества данных не начнётся.